Predicting the Subway Pulse

A Machine Learning Study of NYC Transit: The Issues of Overcrowding and Fare Evasion

Ashley Tewari (ant5149) Sheetal Sood (ss21040)

Abstract

The time and monetary costs of overcrowding and fare evasion problems in the NYC subway transit system are far and wide, and effect millions of people across the city. We use five datasets from the MTA Open Data portal and the NOAA Climate Data in order to implement supervised machine learning models to predict time and locations of overcrowding, and determine predisposed fare evasion risk for subway stations. We use Linear Autoregression, Random Forest, and LSTM models using hourly ridership data for overcrowding. We develop the overcrowding framework as a regression task and a binary crush event classification task. We engineered twenty-five structural features and operational features to train Logistic Regression, Random Forest, and XGBoost classifiers to determine stations as high or low risk. We found that LSTM performed worse than Random Forest for ridership regression, while XGBoost had an almost perfect crush event classification accompanied by weekly lag features. We also found that proportion of entry permitted entrances is the best predictor of fare evasion risk, though only using structural features is insufficient. Further developments in this area would benefit from real time service alert data. We recommend that future studies integrate adjacent station features and action station level evasion rates to improve both overcrowding prediction and fare evasion classification respectively.

1 Introduction

The New York City subway system is one of the busiest transit networks globally. This public transit system serves around three million people everyday. It's scale is massive, and so are its challenges. The subway system experiences both station overcrowding and fare evasion risk.

With machine learning methodology, we can discover predictors and solutions for these pressing issues. These methods and models can learn complex and non-linear patterns from huge datasets and create strong predictions. In this paper we apply supervised learning methods in machine learning to predict when and where overcrowding happens, and identify which stations have fare evasion risk.

To study overcrowding, we will train Linear Autoregression, Random Forest, and LSTM models on hourly ridership data from the busiest NYC subway stations. This develops this issue as a regression task and a binary classification task. To study fare evasion classification, we engineer twenty-five structural and operational features to train Logistic Regression, Random Forest, and XGBoost classifiers. These models will identify high risk stations based on on the characteristics of each station.

Our findings reveal that Random Forest performs better than LSTM on ridership regression, XGBoost achieves an almost perfect crush event classification, and strongest structural predictor of fare evasion risk is the proportion of open and unstaffed entrances.

2 Related Work

Station Overcrowding & Ridership Forecasting Ridership forecast and the overcrowding issue is a prevalent issue that has been studied at large by researchers. Methods in classical learning, machine learning, and deep learning have been used to better understand what drives these problems. Shrivastava et. al [1] studied bus crowding predictions with a Cluster Based LSTM model. The pre-cluster time series models were then trained on electronic ticketing datasets. This is similar to our method of analyzing overcrowding as a time series prediction issue. While Shrivasta et al. use Indian transit data for bus stops, we use MTA hourly data on NYC Subway stations to predict crush events. Additionally, Ma et al. [4] studied the Beijing metro system by using a parallel CNN–BLSTM model that was able to identify spatial and temporal features at the same time. This had better performance than LSTM and ARIMA baseline comparisons. They found that temporal features are superior to spatial features for longer horizon forecasting. This particular insight is what lead to our decision to make lag based temporal encodings

a priority instead of spatial station graph features when implementing our LSTM model. Lastly, Grinsztajn et al. [5] researched the effect of benchmarking tree based models on deep learning models across forty-five tabular datasets. They found that Random Forest and XGBoost implementations had higher performances that neural networks on structured data of a medium size. This was mainly because neural networks have a harder time incorporating irregular target functions and uninformative features. This insight is consistent with our upcoming finding that Random Forest performs better than LSTM on NYC subway ridership data. This is likely due to our data the structure and irregularity of our datasets during certain temporal events.

Fare Evasion & Station Design A vast majority of research on fare evasion risk uses a combination of observational auditing, econometric modeling, and various machine learning methods. Reddy et al. [2] studied the fare evasion in the NYC subway system by using stratified random sampling to also document their measurement framework. They found that fare evasion is highly correlated with station activity, neighborhood income, entrance configuration, and staffed or unstaffed entrances. Specifically, their study revealed that open entrances in general are associated with more fare evasion. These findings motivated us to construct structural risk indicators from entrance metadata for our feature engineering. The metadata includes staffing ratios, entrance type diversity, and proportion of open entry points. A study by Freiria and Sousa [3] analyzed fare evasion using a large-scale spatial regression analysis on bus lines in Lisbon. Their results showed that geographic and locational factors had a significant effect on evasion predictions, inspection frequency, and time-of-day features. Their paper focuses on the influence physical and spatial context have on evasion patterns, and relies on econometric spatial regression. This also required the use of observed inspection logs and the assumption that the predictors have linear associations. For our paper, we chose to implement machine learning classifiers such as Random Forest and XGBoost to uncover non-linear associations from structural features. These models are also able to operate without additional enforcement station-level data. This allows our framework to be better suited for prospective station risk assessments using only infrastructure metadata. Morevoer, Barbino and Ventura [6] studied fare evasion by using a generalized linear regression model compared to an artificial neural network that predicts frequency of fare evasion from Italian bus system datasets. Their findings indicate that the ANN moderately performed better than the generalized linear regression model (the econometric model). Additionally, the found that passenger volume is the superior predictor of fare evasion. In contrast, our paper moves away from operational inspection data, and instead uses structural and physical features at a station level. We decide to frame fare evasion risk as a binary classification issue. This allows us to develop insights that are more readily applicable to the interpretation of policy, while not needing real time data on enforcement at stations.

3 Data

Our five datasets are provided by the MTA Open Data portal and the NOAA Climate Data Online API. All five datasets are used across our modules.

MTA Origin-Destination Ridership Estimates This dataset has information on estimated average ridership for origin and destination pairs for the NYC subway system. We used 200,000 rows from data from 2025 that cover 423 unique origin and 424 unique destination station complexes. Significant areas include timestamp, origin stations, destination stations, geographic coordinates, and estimated average ridership. Our finalized Module dataset includes 8,591 unique origin-destination pairs from 14,023 observations.

MTA Subway Hourly Ridership (2025) This dataset has information on hourly ridership by station complex and fare payment class for either the OMNY or MetroCard services. There are 1 million rows that cover the months January to April in 2025 for 428 unique station complexes. We aggregate across payment methods and have rows that each identify station and hour observations with certain features. The features are as follows:

- **Temporal Features:** hour of day, day of week, weekend indicator, month, sine or cosine encodings for hour and day to visualize cyclical patterns
- **Rush Hour Indicators:** binary flags for morning rush (7am to 9am) and evening rush (4pm to 7pm)
- **Lag Features:** ridership 1, 2, 3, 24, and 168 hours prior, with missing data flags indicating whether each lag represents a real observation or not
- **Rolling Statistics:** 3 hour rolling mean and standard deviation of ridership

- **Weather:** daily TMAX, TMIN, TAVG, PRCP that are merged by date

A *crush event* is any station and hour pair that exceeds the 85th percentile for ridership of that particular station. This gives us a system crush rate of 15.0% for the top five stations. Our dataset is organized by 70% train with 8,551 rows and a crush rate of 14.1%, 15% validation with 1,832 rows and a crush rate of 16.9%, and 15% test with 1,834 rows and a crush rate of 17.4%.

Figure 1 shows

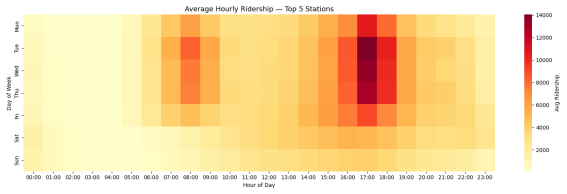

Figure 1: This heatmap shows us the average hourly ridership across the top five busiest stations. The evening rush hour at around 17:00 or 5 PM shows the highest ridership, particularly on weekdays.

MTA Subway Fare Evasion & Entrances This dataset gives information on entrance metadata for 2,120 entrances for 445 station complexes in NYC. Significant fields are entry and exit permissions, staffing statues, geographic coordinates, and entrance types (stairs, elevator, escalator, ramp, and alike). Below are 25 station features we organized into four different categories:

- **Entrance Counts:** total entrances, counts by type (stair, elevator, escalator, ramp, walkway, and alike)
- **Access Ratios:** proportion of entrances with entry allowed, proportion staffed, proportion that is ADA accessible
- **Structural Complexity:** entrance type diversity such as number of distinct entrance types, the geographic spread of entrances in meters
- **Ridership Context:** mean and standard deviation of hourly ridership that is merged from the hourly ridership dataset

Our target label *high_evasion* represents a binary indicator that is constructed from open entrances and exit only counts. We consider these features structural risk indicators and are excluded from our model's feature set so that we do not include circular reasoning. In other words, this is so that we do not have entrance data as both our input and output, leading to the model falsely having high accuracy. We see a balanced label distribution with 347 high risk and 346 low risk stations from a total of 693 stations we assessed.

Figure 2 shows a feature correlation matrix. *num_entrances* and *entrance_type_diversity* are have a high correlation at 0.62, while the label *high_evasion* is most correlated with *num_entrances* at 0.22. This suggests that there is no single feature that is strongly predictive and motivates the use of ensemble machine learning methods.

Figure 2: A feature correlation matrix for information from the fare evasion dataset. The low correlations with our target label motivate the use of ensemble methods over using simple linear models.

MTA Station & Complex Metadata This dataset has information on metadata for 455 subway station complexes, and was used across our Modules. Key features include geographic coordinates, borough, structure type, and ADA status. There are 156 stations for Brooklyn, 121 for Manhattan, 79 for Queens, 68 for the Bronx, and 21 for Staten Island. Structure features are characterized as Subway, Elevated, Open Cut, and alike. These structure features have values of 235, 136, and 37 respectively.

NOAA Climate Data This dataset contains daily weather observations for NYC. It covers 365 days within the year 2025 from the Central Park weather station. Key features in this dataset are maximum temperature, minimum temperature, precipitation, and average temperature. We have combined hourly ridership with this weather data

to act as an external feature for overcrowding predictions in Module 2.

4 Methods

Feature Engineering We use temporal features, lag features, and missing data flags across lag features, rolling statistics, and daily weather features on a 20-feature input matrix. Temporal features include hour, day of week, rush hour indicators, and cyclical sine/cosine encodings. Lag features are ridership at $t-1$, $t-2$, $t-3$, $t-24$, $t-168$, and are derived using timestamps lookup. This is so that only real observations are included in our model. Our rolling statistics includes 3 hour mean and standard deviation. The daily weather features are described as TMAX, TMIN, TAVG, and PRCP.

Dense Window Filtering For our LTSM sequences we filter the model so that all 12 consecutive observations are required to span 18 hours at most. This is essential in preventing the model from being trained on sequences that cover multiple days and have missing data. This allows us to have consistent time frequencies for our input windows.

Figure 3 shows us the missingness flag diagnostic. We see that lag-24 coverage is at 99% and lag-168 coverage is at 93% for all five stations. This confirms that the majority of lag values actually represent real observations. The daily persistence plot on the bottom left shows a high correlation of $r = 0.858$ for the relationship for current ridership and ridership 24 hours prior. This confirms that lag-24 is performing great as an indicator. The data pipeline panel shows that there is minor loss of data from the dense-window filtering (2,446 $\rightarrow$ 2,431 sequences, 99% retained).

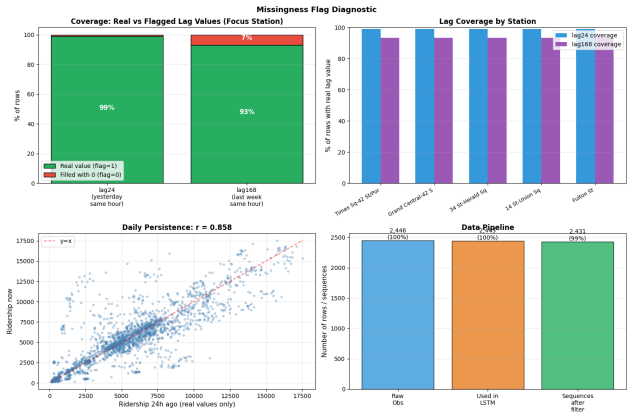

Figure 3: This is the missingness flag diagnostic. From top left: lag coverage at the focus station which is Times Square. From top right: lag coverage across all five stations. From bottom left: daily persistence ($r = 0.858$) confirms that lag-24 is highly predictive. From bottom right: the data pipeline shows very minor loss from the dense-window filtering.

Train, Validation, Test Split We split the data chronologically. This is so that we can prevent major data leakage and respect the dataset's time organization. We chose this method over random splitting because this could cause the model to be trained with future observations rather than real observations.

Regression Models We chose to train three different models based on complexity:

- **Linear Autoregression (our baseline):** this model predicts next hour ridership as a linear combination of the lag features. This is the best fit baseline for time series because it captures persistence patterns with very low complexity.
- **Random Forest:** this model uses 200 trees, a max depth of 12, and is trained on the full flattened feature matrix. This is suitable for our topic because there are non-linear interactions for features such as time, weather, and station specific behaviors. Due to this, the trees are a good option for ridership behavioral patterns.
- **LSTM:** this model uses a two layer architecture of 32 and 16 units, a dropout of 0.30 and 0.25, and batch normalization. It is trained with conditions such as early stopping and learning rate reduction stabilized. We chose this one for the model's natural fit for long-range temporal dependencies for our 12

hour input window. Figure 9 shows us the training curves converging smoothly without any overfitting.

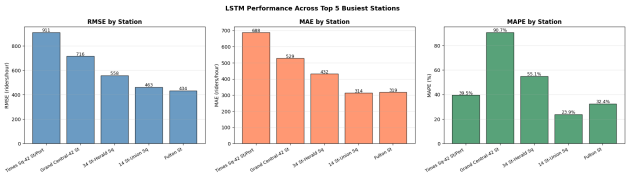

Figure 4: These are our LSTM training curves with Huber loss and MAE. Both of the train and validation metrics converge smoothly. This shows that the model us able to generalize with accuracy even for unseen data.

Classification Models We will consider overcrowding as a binary classification task. We define a *crush event* as a station-hour that exceeds the 85th percentile for ridership. We train Logistic Regression as our baseline, Random Forest, and XGBoost classifiers. These methods are trained with station level lag and temporal features using a time ordered train, validation, and test split.

Adjacent Stations For this study, we have decided not to include adjacent station features. This is due to the potential same timestamp data leakage issues. Additionally, including adjacent station data would lead to higher complexity from the construction of a spatial station graph. This is an area we will consider in future developments.

Evaluation We evaluate regression models with RMSE, MAE, and MAPE. The classification models are evaluated with precision, recall, F1-score, and ROC-AUC. The ROC-AUC will be implemented with the 5 fold cross-validation.

4.1 Module 2: Fare Evasion Classification

Target Construction We construct the binary label *high_evasion* from two different risk indicators. These indicators are the presence of open entrances and the number of exit only gates. In order to avoid circular reasoning, we have excluded both of these indicators from the feature set. This guarantees that the model is able to find structural patterns that are genuine and are not simply recovered from the label construction formula. Due to this, we get a balanced label distribution with 347 high risk stations and 346 low risk stations from 693 stations total.

Feature Set Our model takes 25 features from the entrances dataset and station metadata. These features include entrance counts by type, access ratios (ADA, staffed, entry allowed proportions), structural complexity metrics, and ridership statistics that were merged from the hourly dataset.

Models We train the below five models based on complexity:

- **Majority Class (out baseline):** this consistently predicts the most frequent class, and establishes a lower bound
- **Single-Feature LR (our baseline):** logistic regression on *num_entrances* only, this tests whether station size predicts evasion risk on its own
- **Logistic Regression:** full 25 feature set with balanced class weights. Appropriate as an initial supervised model due to the moderate feature count and the balanced labels
- **Random Forest:** 300 trees, with a max depth of 8, and balanced class weights. This is a best fit for this area due to the entrance features having complex non-linear interactions that trees are able to exploit well
- **XGBoost:** 300 estimators, with a max depth of 6, and a learning rate of 0.1. We chose this for its excellent performance on tabular data and its built-in feature importance

Evaluation These models are evaluated with F1-score and ROC-AUC on a held-out test set. We also use a 5 fold cross-validation F1 reported so as to assess generalizations. We split the dataset at 80/20 by label.

5 Results

5.1 Module 2 Results

Regression Results Our table 1 below shows the regression performance for all three models we used for Times Square. We see that Random Forest greatly outperforms both of our baselines. It reaches an RMSE of 494.6 riders and a MAPE of 7.2%. The LSTM reaches an RMSE of 715.6 and a MAPE of 27.9%. The LSTM therefore underperforms compared to Random Forest despite being the more complex model.

Table 1: Regression results at Times Square.

Model	RMSE	MAE	MAPE%
Linear AR	2009.4	1529.0	73.9
Random Forest	494.6	322.6	7.2
LSTM	715.6	560.3	27.9

Figure 5 shows the hourly ridership forecast for the last 14 days of the test set. Random Forest and LSTM both track the actual ridership closely, while Linear AR fails to capture peak magnitudes. Red shading indicates LSTM-predicted overcrowding events, which align well with actual ridership peaks.

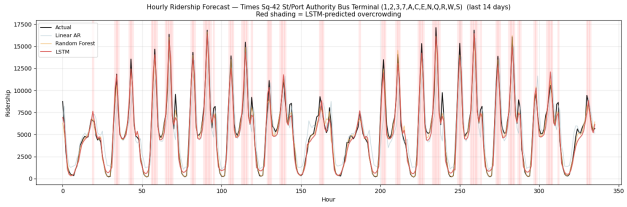

Figure 5: Hourly ridership forecast at Times Square (last 14 days). Random Forest and LSTM closely track actual ridership while Linear AR underestimates peaks. Red shading indicates LSTM-predicted overcrowding events.

Classification Results **Classification Results.** The table 2 below shows the crush event classification results for all three models. We see that XGBoost has the highest performance with a ROC-AUC of 0.996 and an F1 of 0.938. This is followed closely by Random Forest with an AUC 0.994 and an F1 of 0.917, and Logistic Regression with an AUC of 0.984 and an F1 of 0.841).

Table 2: The crush event classification results.

Model	Acc.	Prec.	Recall	F1	AUC
LR	0.941	0.787	0.903	0.841	0.984
RF	0.972	0.950	0.887	0.917	0.994
XGBoost	0.979	0.986	0.893	0.938	0.996

Figure 6 shows us the ROC curves, XGBoost confusion matrix, and feature importance for the crush classification. This confusion matrix showcases 4 false positives and 34 false negatives from a total of 1,834 test observations. The dominant feature is *ridership_lag168* at a weekly pattern. It is followed by cyclical hour encodings.

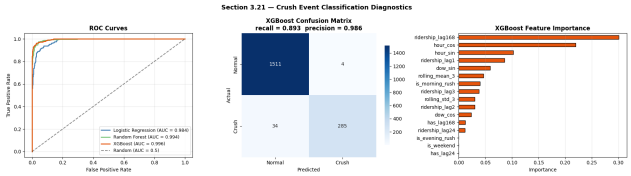

Figure 6: The crush event classification diagnostics. On left: ROC curves for all three models (AUC 0.984 to 0.996). At center: XGBoost confusion matrix (4 false positives and 34 false negatives). On right: XGBoost feature importance, weekly lag dominates.

5.2 Module 2 Results

The below table 5.2 shows the fare evasion classification results. Logistic Regression is able to reach the best test F1 with 0.606 and cross-validation F1 with 0.600. XGBoost reaches the best CV F1 with 0.615. All three of the models outperform the single feature baseline of 0.486. However, they still fall below the majority class F1 of 0.667 This potentially indicates that the problem at hand is simply a very complex and challenging one.

begintable[H]

Model	F1	CV F1
Majority Class	0.667	–
Single-Feature LR	0.486	–
Logistic Regression	0.606	0.600
Random Forest	0.580	0.583
XGBoost	0.570	0.615

Figure 7 shows us the full model diagnostics. The ROC curves are very tightly clustered with from AUC 0.665 to 0.693. This confirms that the models have similar performance. The Logistic Regression confusion matrix is able to correctly identify 82 of 87 low risk stations. However, it misses 47 of 87 high risk stations. The top predictive feature is *pct_entry_allowed* by a large margin. This is followed by *num_ramp* and *std_hourly_ridership*.

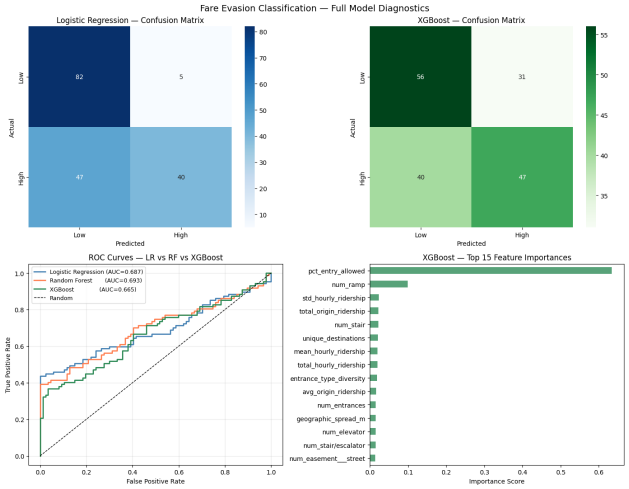

Figure 7: The fare evasion classification diagnostics. On top: confusion matrices for Logistic Regression and XGBoost. On bottom left: ROC curves (AUC 0.665–0.693). On bottom right: XGBoost feature importance, *pct_entry_allowed* dominates.

6 Analysis

6.1 Module 2 Analysis

Random Forest versus LSTM on Regression. LSTM is supposed to be capable of higher performances for sequential data. However, we found that Random Forest reaches a much lower RMSE at 494.6, compared to LSTM's 715.6. Additionally, we see Random Forest with a MAPE of 7.2% while LSTM has a MAPE of 27.9%. We hypothesize that this is happening for a few reasons. Primarily, we know that the dataset has 2,431 LSTM sequences. This is a fairly small amount for a deep learning model that has 30,625 trainable parameters. On the other hand, Random Forest performs the best with tabular data at various sample sizes. Additionally, the most predictive features (*ridership_lag168* and cyclical hour encodings) are tabular features. Tree-based model use these features directly. In contrast, LSTM has to learn these types of patterns implicitly from the sequence structure.

Error Analysis by Hour We can see on Figure 8 that there is a systematic pattern in prediction errors for all of our models. We see that linear AR fails during morning (7 am to 9 am with 800 riders) and evening (5 pm to 7 pm with 500 riders) rush events. This confirms that only using persistence based forecasting does not capture sharp ridership spikes in Times Square specifically. Both the random forest and LSTM models have errors that are lower than one thousand riders for all hours of the day. However, LSTM shows higher errors during the transition events (6am to 8 am and 9pm to 10 pm). Ridership behavior during these times is very irregular.

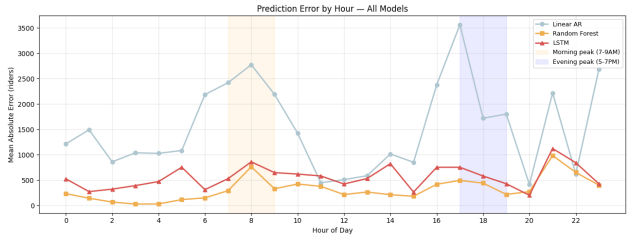

Figure 8: Prediction error by hour of day across all three models. Linear AR fails at rush hours while Random Forest and LSTM maintain consistent low error. Shading indicates morning and evening peak windows.

Cross-Station Performance We can see LSTM performance for the top five busiest stations on Figure 9. Our findings show that times Square is the hardest to predict (RMSE 911, MAPE 39.5%), while 14 St-Union Sq is the easiest (RMSE 463, MAPE 23.9%). Grand Central has a notable anomaly with a moderate RMSE of 716 but a high MAPE of 90.7%. Thus, the model appears to struggle with low ridership hours since small absolute errors can lead to large percentage errors. In order to improve performance for these complex cases, station specific models or additional features that capture station operational characteristics could be beneficial.

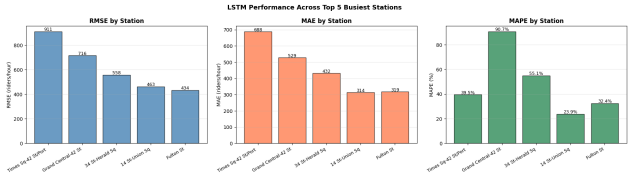

Figure 9: LSTM performance across the top 5 busiest stations. Times Square is hardest to predict (MAPE 39.5%) while 14 St-Union Sq is easiest (MAPE 23.9%). Grand Central shows high MAPE but moderate RMSE.

Crush Event Classification Classification results show XGBoost with an AUC of 0.996 and a F1 of 0.938. These insights show that crush events are highly predictable as temporal and lag features. The dominant feature *ridership_lag168*, which is weekly lag, indicates that stations may follow regular weekly patterns. This makes it so that classification is much easier to do than a regression task. There are 34 false negatives that indicate missed

crush events. These cases are likely due to special circumstances like holidays or construction, and should be looked into further.

6.2 Module 2 Analysis

Majority Class Baseline Struggle All models achieve F1 scores that are below the majority class baseline with a value of 0.667. We hypothesize that this could be due to the target label and the available features. Primarily, we see that the target label is actually a proxy that is constructed from structural indicators. By not using actual observed evasion rates, noise is introduced to the supervision signal. Secondly, entrance counts, structure type, and ridership patterns may be affected by outside factors. These factors are socioeconomic class, enforcement behavior, and fare levels. Therefore, the actual reasons for evasion behavior may not be captured.

Logistic Regression versus Tree Methods While Logistic Regression is the simplest model, it actually achieves an F1 score of 0.606, which is the best of the study. XGBoost ends up with the best CV F1 of 0.615. We believe that the dataset does not have enough data for more complex models to be able to generalize. Both models might be overfitting to the training patterns, which do not hold on the test set.

Feature Importance Findings We see that the dominant feature is *pct_entry_allowed* represents the proportion of entrances where entry is permitted. This feature indicates that stations that have more restricted entry points actually have lower evasion rates. These are areas riders have to pass through certain controlled access points either staffed or not. Thus, targeted enforcement or certain physical changes at stations with more open entrances could potentially reduce fare evasion risk. The importance of *num_ramp* and *std_hourly_ridership* also suggests that stations with high variability in ridership and accessible ramp entrances have possibly higher fare evasion risk.

Misclassified Stations There is an asymmetry in that 82 of the 87 low risk stations are accurately identified, while only 40 of the 87 high risk stations are identified. This reveals a recall of 94% for low risk stations, and a recall of 46% for high risk stations. Thus, the model tends to default to identifying stations as low risk with ambiguous data or features. The 47 high risk stations that were misidentified as low risk may have had characteristics of low risk stations, but accompanied by unmeasured behavioral or enforcement factors. With XGBoost we get the top 20 highest risk stations, which are stations that have high entrance counts, diverse entrance types, and high variability in ridership.

7 Conclusion

This paper presents two machine learning based analyses of NYC subway operations. We address overcrowding prediction and fare evasion risk classifications using datasets provided by the MTA and NOAA.

For overcrowding prediction, we see that XGBoost has the best crush event classification. Random Forest had better performance than LTSM on regression. We know that the NYC subway ridership follows a regular pattern based on the weekly lag features. All models struggle with predictions during transition events for rush hours. Thus, external factors like holidays and station construction could be affecting these prediction cases.

For fare evasion, *pct_entry_allowed* is the strongest structural predictor of evasion risk. This has policy implications for stations where targeted enforcement at stations with open entrances may help mitigate fare evasion. The model has moderate performance which suggests that structural features by themselves are insufficient. Prediction quality would be greatly improved by including actual observed evasion rates.

Limitations There are several limitations in effect. The fare evasion label is actually a proxy rather than a representation of observed evasion rates. Our overcrowding analysis is restricted to only the top five stations, and as previously stated, excludes adjacent station features. We also removed a cross borough clustering module to discover optimal Brooklyn-Queens routes due to course scope and time constraints.

Future Work We have many considerations for future work in this area. Firstly, we believe the LTSM could be improved by the use of real time service alert data and additionally adjacent station features. Our fare evasion model would also benefit from real station level evasion rates. The MTA has data on this, though it is new and not in as much abundance as datasets we used during this study. We suggest that future work also consider cross-borough travel (Brooklyn-Queens) efficiency as a potential project scope with origin and destination clustering methods.

References

[1] Shrivastava, Arpit, et al. "Deep-Learning-Based Model for Prediction of Crowding in a Public Transit System." *Public Transport*, vol. 16, no. 2, June 2024, pp. 449–84. https://doi.org/10.1007/s12469-024-00360-z

[2] Reddy, Alla V., et al. "Measuring and Controlling Subway Fare Evasion: Improving Safety and Security at New York City Transit Authority." *Transportation Research Record*, vol. 2216, no. 1, January 2011, pp. 85–99. https://doi.org/10.3141/2216-10

[3] Freiria, Susana, and Nuno Sousa. "Determinants of Fare Evasion in Urban Bus Lines: Case Study of a Large Database Considering Spatial Components." *Urban Science*, vol. 9, no. 6, June 2025, p. 231. https://doi.org/10.3390/urbansci9060231

[4] Ma, Xiaolei, et al. "Parallel Architecture of Convolutional Bi-Directional LSTM Neural Networks for Network-Wide Metro Ridership Prediction." *IEEE Transactions on Intelligent Transportation Systems*, vol. 20, no. 6, June 2019, pp. 2278–88. https://doi.org/10.1109/TITS.2018.2867042

[5] Grinsztajn, Léo, et al. *Why Do Tree-Based Models Still Outperform Deep Learning on Tabular Data?* July 2022. https://doi.org/10.48550/arxiv.2207.08815

[6] Barabino, Benedetto, and Roberto Ventura. "Comparing Econometric and Machine Learning Models to Fare Evasion Prediction." *Transportation Research Interdisciplinary Perspectives*, vol. 34, no. 101636, November 2025. https://doi.org/10.1016/j.trip.2025.101636